浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Introduction to Programming 2

IBI 1, Lecture 4.2

Dr Rob Young – robert.young@ed.ac.uk

Semester 2, 2025/26

Now…

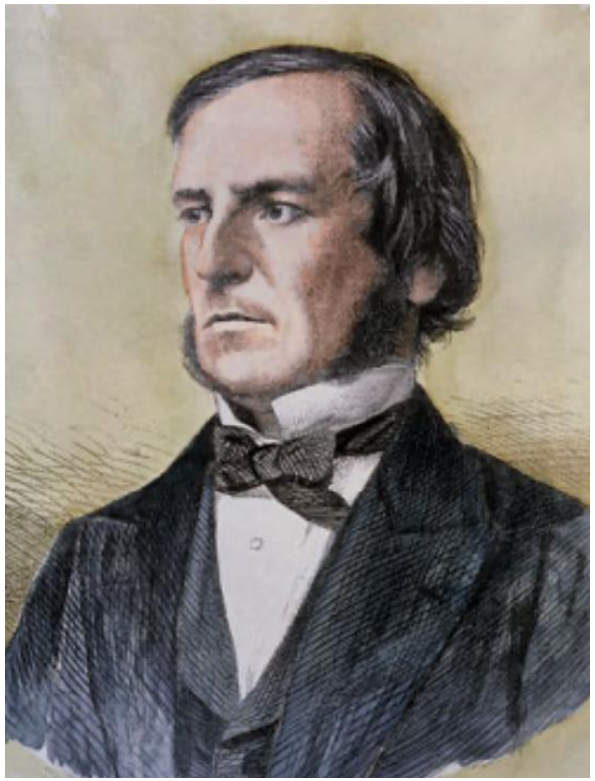
- Define re-usable functions.
- Count and keep track of numbers.
- Compare things to other things.
- Include decision points.
- Repeat things until something happens.

Outline

1. Booleans
2. If statements
3. Loops
4. Paired programming

Introduction to Booleans

- Boolean variables (= Booleans) are special variables.
- They can take only two values, "True" and "False".
- Booleans are useful when something needs to be checked to determine how the programme continues, e.g. check whether there is a turn in the track or whether it goes straight, adjust running direction accordingly.



George Boole

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

```
a < b
```

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

```
a < b
```

- Is a smaller than or equal to b?

```
a <= b
```

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

  `a < b`
- Is a smaller than or equal to b?

  `a <= b`
- Is a greater than or equal to b?

  `a >= b`

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

  `a < b`
- Is a smaller than or equal to b?

  `a <= b`
- Is a greater than or equal to b?

  `a >= b`
- Is a equal to b?

  `a == b`

  Why two equal signs?

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

  `a < b`
- Is a smaller than or equal to b?

  `a <= b`
- Is a greater than or equal to b?

  `a >= b`
- Is a equal to b?

  `a == b`

  One equal sign would assign the value of variable b to a

Comparing values

- Booleans are often used for comparison.
- Is a smaller than b?

  `a < b`
- Is a smaller than or equal to b?

  `a <= b`
- Is a greater than or equal to b?

  `a >= b`
- Is a equal to b?

  `a == b`

  One equal sign would assign the value of variable b to a
- Is a not equal to b?

  `a != b`

Truth tables

Booleans can be combined and modified using not, and, and or. If A is True, then not A is False, and the other way round. The meanings of and and or can be seen from these truth tables.

Truth tables

Booleans can be combined and modified using not, and, and or. If A is True, then not A is False, and the other way round. The meanings of and and or can be seen from these truth tables

And		
A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Truth tables

Booleans can be combined and modified using not, and, and or. If A is True, then not A is False, and the other way round. The meanings of and and or can be seen from these truth tables.

And		
A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Or		
A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Exercises

Determine the truth of the following statements

- `not (4>0)`
- `1 <= 1`
- `(5 < 7) and (12 < 4)`
- `3*4 == 12 or 2 < 5`
- `x = 5`

```
y = 5
x == y
```

Different kinds of "or"

Sometimes, we need "either x or y" (i.e. x or y, but not both). What would the truth table for this look like? Can you write a line of Python code to express this?

Exercises

Determine the truth of the following statements

- `not (4>0)` → `False`
- `1 <= 1` → `True`
- `(5 < 7) and (12 < 4)` → `False`
- `3*4 == 12 or 2 < 5` → `True`
- `x = 5`

`y = 5`

`x == y` → `True`

Different kinds of "or"

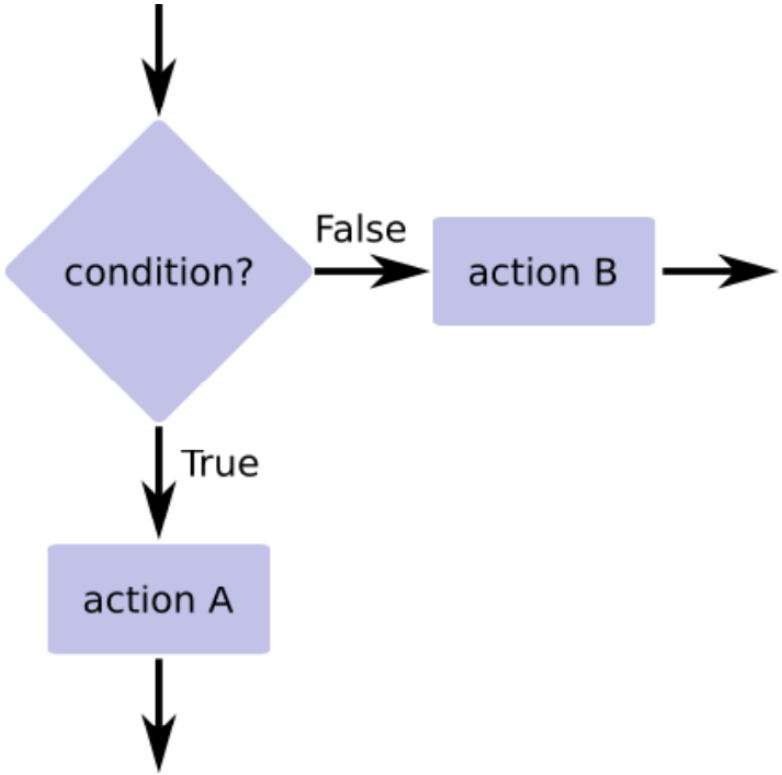
Sometimes, we need "either x or y" (i.e. x or y, but not both). What would the truth table for this look like? Can you write a line of Python code to express this? `(x == True or y == True) and x != y`

Outline

1. Booleans
2. If statements
3. Loops
4. Paired programming

If statements

If-statements are used when a decision has to be made before the programme goes on.



If statements

If

```
if x%2 == 0:
      print ("x is even")
```

- `print()` – show something on the screen.
- All the lines that are indented belong to the same `if` statement (in this case, just one line).

If statements

If

```
if x%2 == 0:
     print ("x is even")
```

- `print()` – show something on the screen.
- All the lines that are indented belong to the same `if` statement (in this case, just one line).

What would this return for x=4, x=3 and x=3.14?

If statements

If

```
if x%2 == 0:
     print ("x is even")
```

- `print()` – show something on the screen.
- All the lines that are indented belong to the same `if` statement (in this case, just one line).

What would this return for x=4, x=3 and x=3.14?

True, False, False

If statements

If – Else

```
if x%2 == 0:
      print ("x is even")
else:
      print("x is odd")
```

If statements

If – Else

```
if x%2 == 0:
     print ("x is even")
else:
     print("x is odd")
```

If – Elif (Else if) – Else

```
if x%2 == 0:
     print ("x is even")
elif x%2 == 1:
     print("x is odd")
else:
     print("x is not an integer")
```

If statements

If – Else

```
if x%2 == 0:
     print ("x is even")
else:
     print("x is odd")
```

If – Elif (Else if) – Else

```
if x%2 == 0:
     print ("x is even")
elif x%2 == 1:
     print("x is odd")
else:
     print("x is not an integer")
```

What would this return for x=4, x=3 and x=3.14?

If statements

If – Else

```
if x%2 == 0:
     print ("x is even")
else:
     print("x is odd")
```

If – Elif (Else if) – Else

```
if x%2 == 0:
     print ("x is even")
elif x%2 == 1:
     print("x is odd")
else:
     print("x is not an integer")
```

What would this return for x=4, x=3 and x=3.14?

Even, odd, not an integer

If Statements

Collatz sequence

For any integer c_n, the next number n the Collatz sequence c_{n+1} is defined as:

$$c_{n+1} \begin{cases} \frac{c_n}{2} & \text{if cn is even} \\ 3 * c_n + 1 & \text{if cn is odd} \end{cases}$$

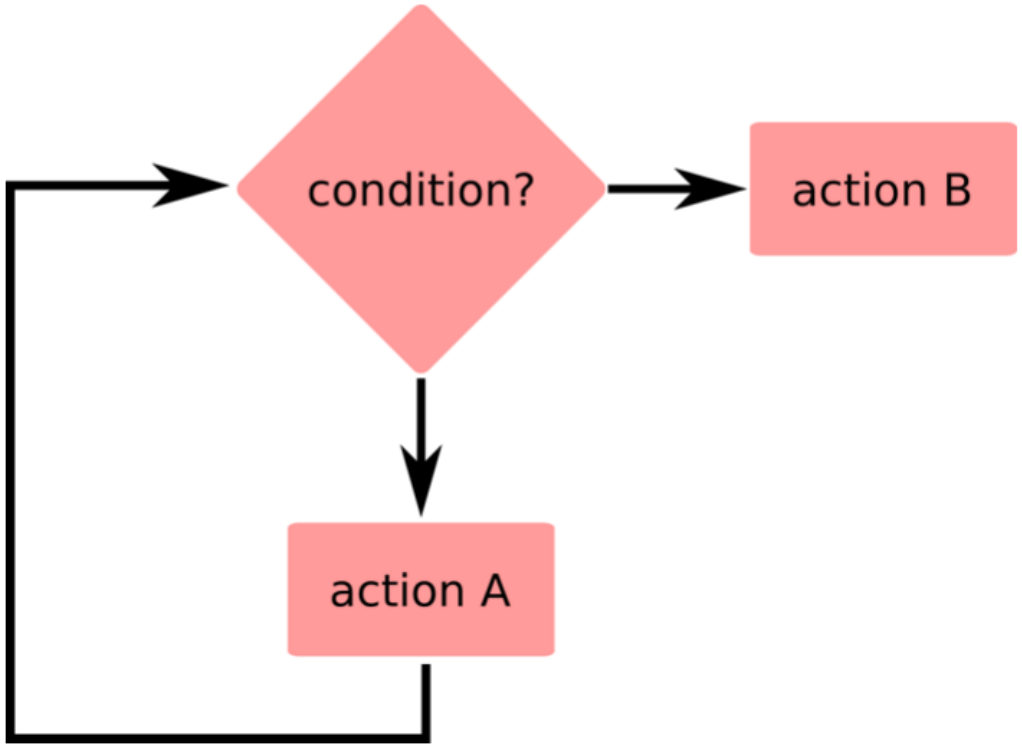
Can you write a piece of code that takes any integer c_n and computes the next number in the Collatz sequence?

Outline

1. Booleans
2. If statements
3. Loops
4. Paired programming

Loops

- Loops allow users to repeat the same thing many times.
- Repetition can happen either a pre-determined number of times or until a pre-determined condition is fulfilled.

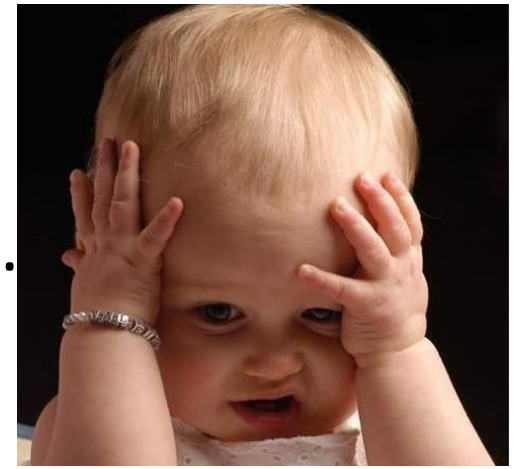


For-loop

- Do something a specific number of times.
- e.g. count down from 25:

```
count=25
print(count)
for i in range(1,25):
    count=count-1
    print("count: ", count)
```

- `range(a,b)` – numbers from a (included) to b (excluded).
- All the lines that are indented belong to the same for statement (two in this case).



For-loop

Excercise

What is the output of this piece of code?

```
for j in range(0,10):
     print("2*", j, " is ", 2*j)

print(":)")
```

For-loop

Excercise

What is the output of this piece of code?

```
for j in range(0,10):
     print("2*", j, " is ", 2*j)

print(":)")
```

2*0 is 0, 2*1 is 1,…, 2*9 is 18

:)

While-loop

- Do something until something happens.

While-loop

- Do something until something happens.
- e.g. draw random numbers until a specific number is drawn.

```
import random
j=0
while j!= 23:
      j = random.randint(1,100)
      print(j)
```

- random – library for random numbers
- randint(a,b) – draw random integer between a and b

While-loop

- Alternative:

```
import random
while 1==1:
	j = random.randint(1,100)
	print(j)
	if j==23:
		break
```

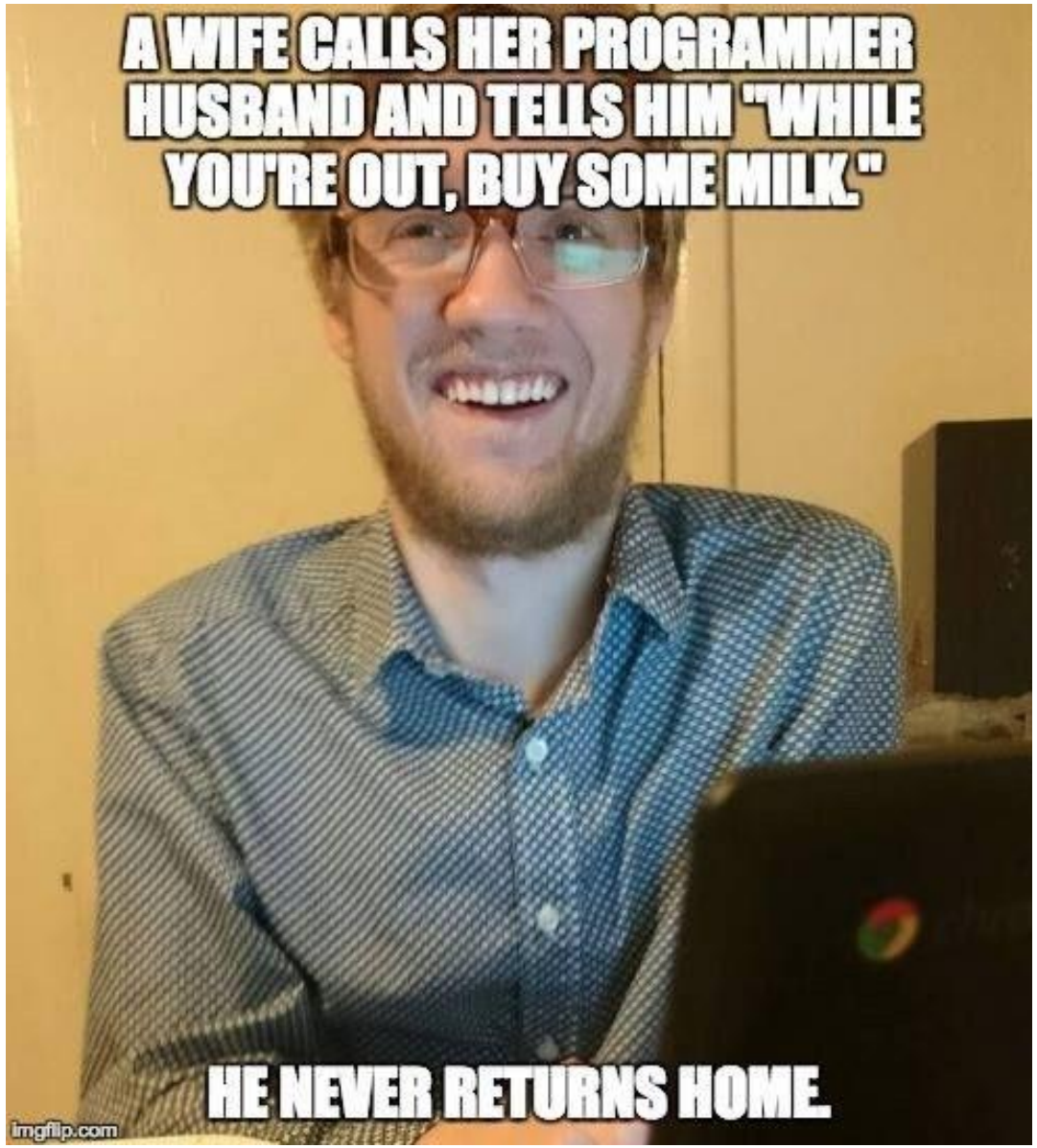
While-loop

- Alternative:

```
import random
while 1==1:
	j = random.randint(1,100)
	print(j)
	if j==23:
	break
```

- Why are "while" loops dangerous?

While-loop


A WIFE CALLS HER PROGRAMMER HUSBAND AND TELLS HIM "WHILE YOU'RE OUT, BUY SOME MILK."
HE NEVER RETURNS HOME
imgflip.com

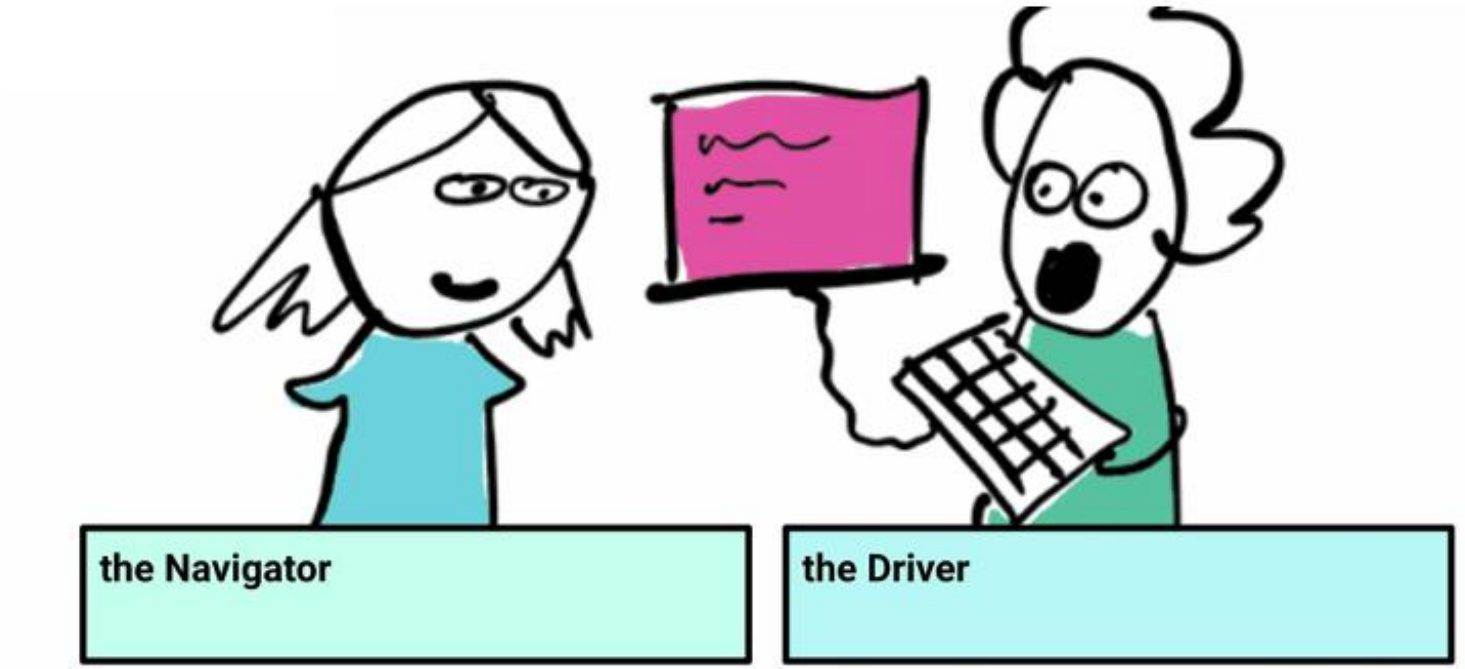
Outline

1. Booleans
2. If statements
3. Loops
4. Paired programming

Paired programming

- The driver: types at the keyboard, explaining what they are doing
- The navigator: helps out, looking for bugs, asking questions

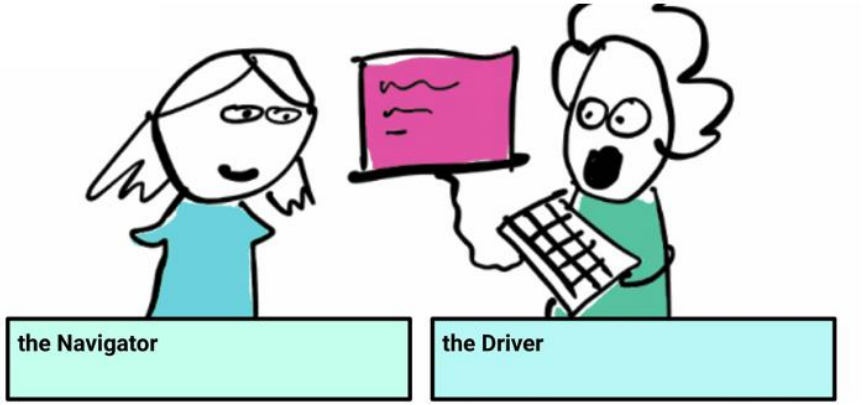
Paired programming



- After 15 mins, swap over!

- We will have a go at this during this week's practical – please arrive on time.

Advantages of paired programming



- Learn useful problem-solving, debugging skills as well as the course material.
- Enables students to work with different partners.
- Learn different approaches to solving the same problems.

Learning objectives

Now, you should be able to:

- Plan a project using pseudocode.
- Explain how variable work within a piece of code.
- Explain the concept of loops.
- Understand and use Booleans.

浙江大学爱丁堡大学联合学院

ZJU-UoE Institute

Introduction to Programming 2

Any questions?

IBI 1, Lecture 4.2

Dr Rob Young – robert.young@ed.ac.uk

Semester 2, 2025/26